

Week-6 Practical — BookFlow Library API [K. Priyanka, 2520090001]

Task : BookFlow Library API — Modular Routing and Book Management

Week_6_BookFlow_project/

├── main.py

└── routes/

 ├── __init__.py

 ├── catalog.py

 └── admin.py

routes/catalog.py code:

```
from fastapi import APIRouter, HTTPException
```

```
router = APIRouter()
```

```
books = [
```

```
{
```

```
    "id": 101,
```

```
    "title": "Python Programming",
```

```
    "author": "John Smith",
```

```
    "price": 450,
```

```
    "isbn": "9781234567890",
```

```
    "status": "Available"
```

```
},
```

```
{
```

```
    "id": 102,
```

```
    "title": "Database Management",
```

```
    "author": "Robert Brown",
```

```
    "price": 550,
```

```
    "isbn": "9789876543210",
```

```
    "status": "Available"
```

```
}
```

```
]
```

```

@router.get("/books")
def get_books():
    return books

@router.get("/books/{book_id}")
def get_book(book_id: int):
    for book in books:
        if book["id"] == book_id:
            return book
    raise HTTPException(
        status_code=404,
        detail="Book not found"
    )

```

routes/admin.py code:

```

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from routes.catalog import books

router = APIRouter()

class Book(BaseModel):
    title: str
    author: str
    price: float = Field(ge=0)
    isbn: str
    status: str

@router.post("/admin/books")
def add_book(book: Book):
    new_id = max([b["id"] for b in books], default=100) + 1
    new_book = {

```

```

        "id": new_id,
        book.model_dump()
    }
    books.append(new_book)
    return {
        "message": "Book added successfully",
        "book": new_book
    }

```

```

@router.put("/admin/books/{book_id}")
def update_book(book_id: int, book: Book):
    for index, existing_book in enumerate(books):
        if existing_book["id"] == book_id:
            updated_book = {
                "id": book_id,
                book.model_dump()
            }
            books[index] = updated_book
            return {
                "message": "Book updated successfully",
                "book": updated_book
            }
    raise HTTPException(
        status_code=404,
        detail="Book not found"
    )

```

```

@router.delete("/admin/books/{book_id}")
def delete_book(book_id: int):
    for index, book in enumerate(books):
        if book["id"] == book_id:
            deleted_book = books.pop(index)

```

```

    return {
        "message": "Book deleted successfully",
        "book": deleted_book
    }

raise HTTPException(
    status_code=404,
    detail="Book not found"
)

```

main.py code:

```

from fastapi import FastAPI

from routes.catalog import router as catalog_router

from routes.admin import router as admin_router

app = FastAPI(
    title="BookFlow Library API"
)

app.include_router(catalog_router)
app.include_router(admin_router)

```

GET default all the data:

127.0.0.1:8000/docs#/default/get_books_books_get

default

GET /books Get Books

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/books' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/books

Server response

Code Details

200

Response body

```
{
  "id": 101,
  "title": "Python Programming",
  "author": "John Smith",
  "price": 100,
  "isbn": "9781234567890",
  "status": "Available"
},
{
  "id": 102,
  "title": "Database Management",
  "author": "Robert Brown",
  "price": 150,
  "isbn": "9780987654321",
  "status": "Available"
}
```

Response headers

```
content-length: 249
content-type: application/json
date: Tue, 15 Sep 2020 06:05:14 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

GET:

GET

/books/{book_id}

Get Book

Parameters

Cancel

Name	Description
book_id ^{required}	
Integer (path)	101

Execute

Response:

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8080/books/101' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8080/books/101
```

Server response

Code	Details
200	Response body <pre>{ "id": 101, "title": "Python Programming", "author": "Sam Saini", "price": 499, "isbn": "9781234567890", "status": "Available" }</pre> Response headers <pre>content-length: 117 content-type: application/json date: Tue, 15 Sep 2026 06:22:00 GMT server: nginx</pre>

Responses

Code	Description	Links
200	Successful Response	No links

POST:

POST

/admin/books

Add Book

Parameters

Cancel

Reset

No parameters

Request body ^{required}

application/json

Edit Value

Schema

```
{
  "title": "Java Programming",
  "author": "Priyanka",
  "price": 600,
  "isbn": "9781234567891",
  "status": "Available"
}
```

Execute

Response:

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/admin/books' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Java Programming",
    "author": "Priyanka",
    "price": 600,
    "isbn": "9781234567891",
    "status": "Available"
  }'
```

Request URL

```
http://127.0.0.1:8080/admin/books
```

Server response

Code	Details
200	Response body <pre>{ "message": "Book added successfully", "book": { "id": 102, "title": "Java Programming", "author": "Priyanka", "price": 600, "isbn": "9781234567891", "status": "Available" } }</pre> Response headers <pre>content-length: 160 content-type: application/json date: Tue, 15 Sep 2026 06:24:38 GMT server: nginx</pre>

Responses

Code	Description	Links
200	Successful Response	No links

PUT:

PUT

/admin/books/{book_id} Update Book

CancelReset

Parameters

Name	Description
book_id * required	
integer (path)	101

Request body * required

application/json

Edit ValueSchema

```
{  "title": "JavaScript Essentials",  "author": "John Smith",  "price": 499,  "isbn": "9781234567890",  "status": "Available"}  }
```

Execute

Response:

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8080/admin/books/101' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "title": "JavaScript Essentials",
    "author": "John Smith",
    "price": 499,
    "isbn": "9781234567890",
    "status": "Available"
  }'
```

Request URL

http://127.0.0.1:8080/admin/books/101

Server response

CodeDetails

200

Response body

```
{  "message": "Book updated successfully",  "book": {    "id": 101,    "title": "JavaScript Essentials",    "author": "John Smith",    "price": 499,    "isbn": "9781234567890",    "status": "Available"  }  }
```

Download

Response headers

```
content-length: 365
content-type: application/json
date: Thu, 10 Sep 2026 08:28:58 GMT
server: nginx
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header

DELETE:

DELETE

/admin/books/{book_id} Delete Book

Cancel

Parameters

Name	Description
book_id * required	
integer (path)	102

Execute

Response:

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8080/admin/books/102' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8080/admin/books/102

Server response

CodeDetails

200

Response body

```
{  "message": "Book deleted successfully",  "book": {    "id": 102,    "title": "Database Management",    "author": "Robert Brown",    "price": 199,    "isbn": "9789876543210",    "status": "Available"  }  }
```

Download

Response headers

```
content-length: 367
content-type: application/json
date: Thu, 10 Sep 2026 08:30:35 GMT
server: nginx
```

Responses

Code	Description	Links
200	Successful Response	No links